Description et responsabilité du microservice

Review Service · Étudiant 3 · avis et notes entre membres de la plateforme SkillSwap

Rôle

Gère les avis (note de 1 à 5 et commentaire) laissés par un membre sur un autre membre après un échange de compétences terminé. L'authentification reste dans Keycloak.

Responsabilités

- Créer un avis après un échange terminé (note 1 à 5, un seul avis par échange et par auteur)
- Consulter un avis par id, ou lister les avis d'un membre avec pagination
- Calculer la note moyenne et le nombre d'avis d'un membre
- Modifier un avis (auteur uniquement) ; supprimer (auteur ou ADMIN)
- Consommer ExchangeCompleted et UserDeleted ; publier ReviewCreated et ReviewDeleted
- Renvoyer des erreurs claires (400, 403, 404, 409) au format Problem Details

Fiche technique

Eureka : REVIEW-SERVICE

Port : 8085

Base : H2 privée (aucune clé étrangère vers un autre service)

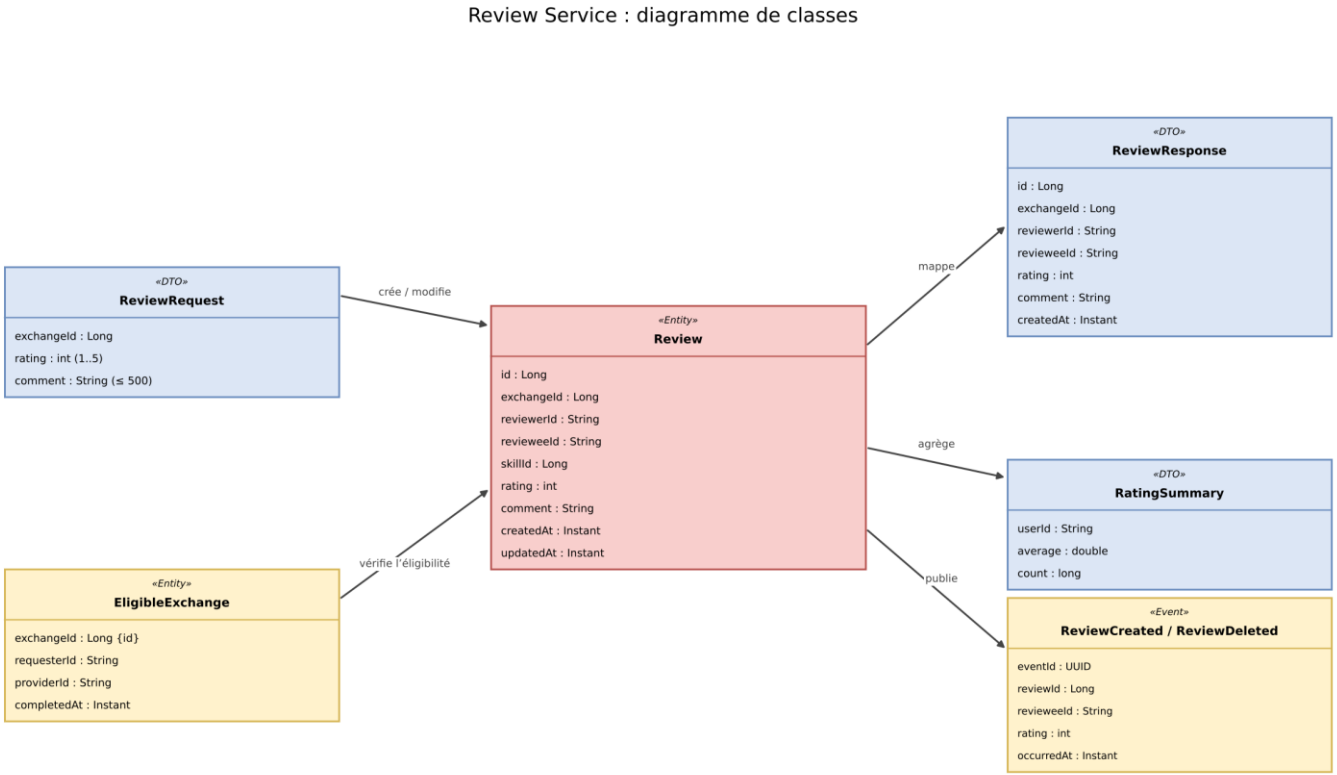
Messagerie : RabbitMQ (producteur et consommateur)

Sécurité : JWT Keycloak (USER, ADMIN)

Accès : via la Gateway /api/reviews/**

API documentée avec Swagger

Diagramme de classes du microservice



Principaux endpoints REST et codes HTTP

Via l'API Gateway (port 8080) · authentification par JWT Keycloak · rôles USER et ADMIN

Méthode	Endpoint	Description	Codes HTTP
POST	/api/reviews	Créer un avis (échange terminé)	201, 400, 403, 409
GET	/api/reviews/{id}	Consulter un avis	200, 404
GET	/api/reviews?userId=&page=&size=	Lister les avis d'un membre (paginé)	200, 400
GET	/api/reviews/users/{userId}/average	Note moyenne d'un membre	200, 404
PUT	/api/reviews/{id}	Modifier (auteur uniquement)	200, 400, 403, 404
DELETE	/api/reviews/{id}	Supprimer (auteur ou ADMIN)	204, 403, 404

200 OK · 201 Created · 204 No Content · 400 requête invalide · 401 non authentifié · 403 accès interdit (échange non éligible ou non auteur) · 404 introuvable · 409 conflit (avis déjà déposé pour cet échange)

Événements RabbitMQ

Exchange topic : skillswap.events · files durables · ack manuel

Consommés par Review Service

exchange.completed (ExchangeCompleted)

→ exchange-service : un échange est terminé

→ review-service : l'échange devient éligible à un avis

user.deleted (UserDeleted)

→ user-service : un profil est supprimé

→ review-service : supprime les avis de ce membre

Publiés par Review Service

review.created (ReviewCreated)

→ notification-service : prévient le membre noté

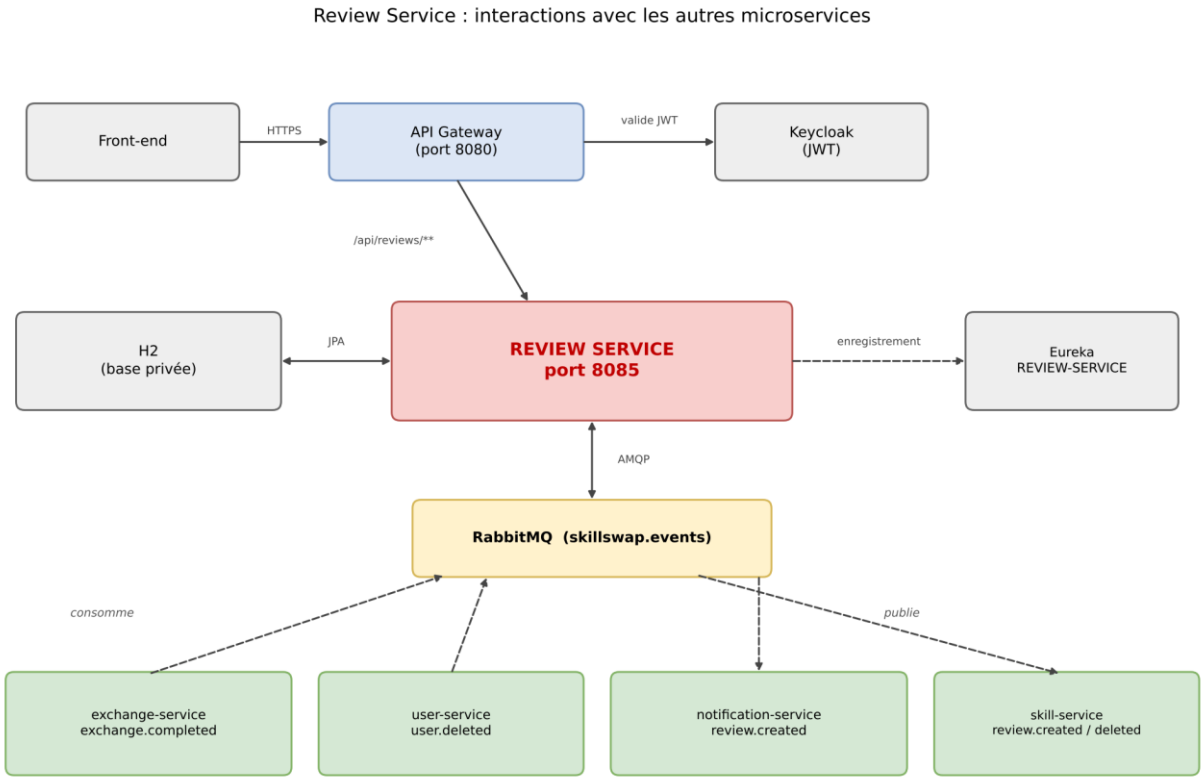
→ skill-service : recalcule la note moyenne de la compétence

review.deleted (ReviewDeleted)

→ skill-service : recalcule la note moyenne

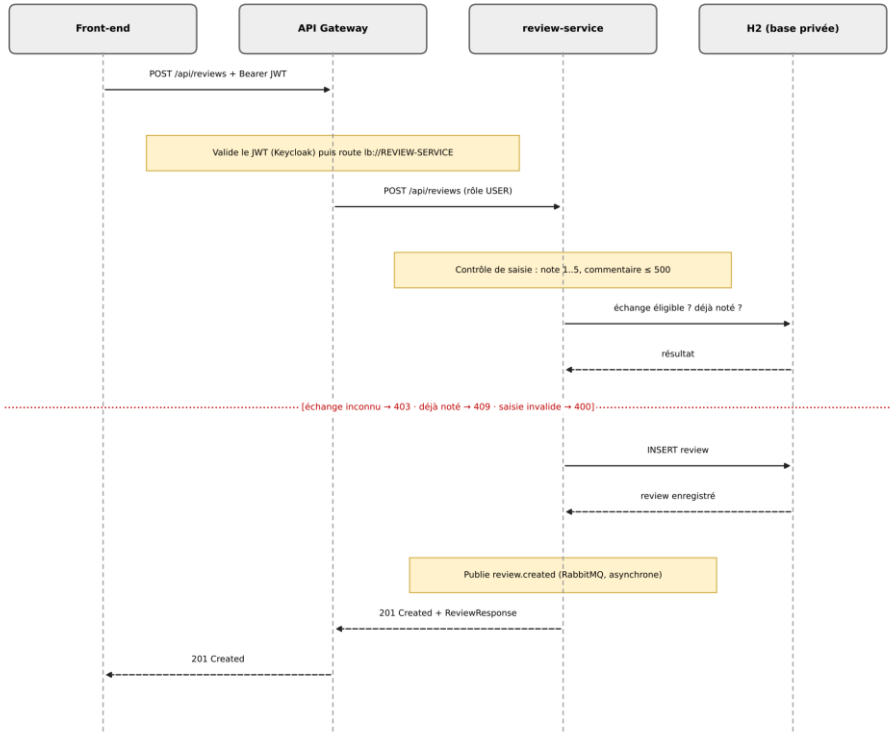
→ aucun autre consommateur

Interactions avec les autres microservices



Exemple de communication synchrone (REST)

Le Front-end appelle POST /api/reviews du review-service via l'API Gateway (REST)



Exemple de communication asynchrone (RabbitMQ)

exchange-service publie exchange.completed ; review-service publie review.created, consommé par notification et skill

